EXPERIMENT-3

NAME: INIYAN V ROLL NO:240701195

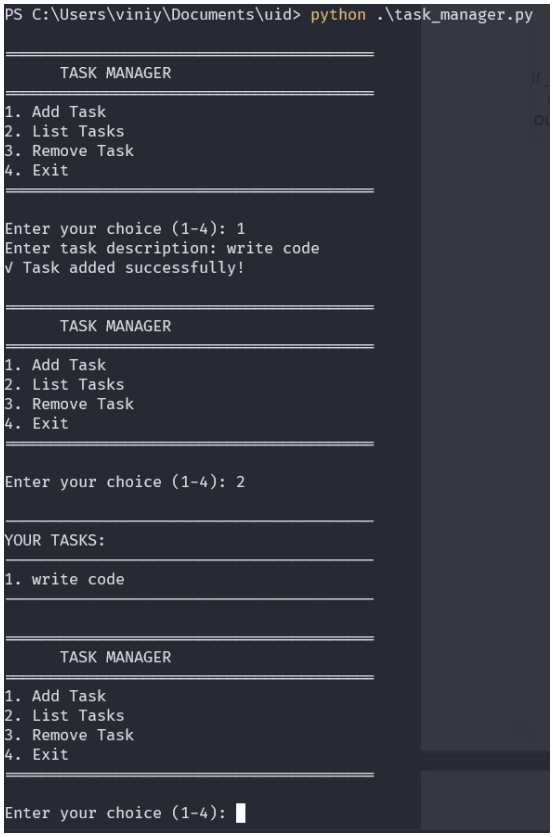
1.**COMMAND LINE INTERFACE:**

PYTHON CODE:

```
def display_menu():
    print("\n" + "="*40)
    print("     TASK MANAGER")
    print("="*40)
    print("1. Add Task")
    print("2. List Tasks")
    print("3. Remove Task")
    print("4. Exit")
    print("="*40)


def add_task(task_list):
    task = input("Enter task description: ").strip()
    if task:
        task_list.append(task)
        print(f"✓ Task added successfully!")
    else:
        print("✗ Task description cannot be empty.")


def list_tasks(task_list):
    if not task_list:
        print("\nNo tasks found. Your task list is empty!")
    else:
        print("\n" + "-"*40)
        print("YOUR TASKS:")
        print("-"*40)
        for index, task in enumerate(task_list, start=1):
            print(f"{index}. {task}")
        print("-"*40)


def remove_task(task_list):
    if not task_list:
        print("\nNo tasks to remove. Your task list is empty!")
        return

    list_tasks(task_list)
    try:
        task_num = input("\nEnter task number to remove: ").strip()
        task_index = int(task_num) - 1

        if 0 <= task_index < len(task_list):
            removed_task = task_list.pop(task_index)
            print(f"✓ Removed: '{removed_task}'")
        else:
```

```
        print(f"✗ Invalid task number. Please enter a number between 1 and {len(task_list)}.")
    except ValueError:
        print("✗ Invalid input. Please enter a valid number.")


def main():
    tasks = []
    while True:
        display_menu()
        choice = input("\nEnter your choice (1-4): ").strip()

        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            list_tasks(tasks)
        elif choice == "3":
            remove_task(tasks)
        elif choice == "4":
            print("\nThank you for using Task Manager. Goodbye!")
            break
        else:
            print("\n✗ Invalid choice. Please enter a number between 1 and 4.")


if __name__ == "__main__":
    main()
```

OUTPUT:



2. **GRAPHICAL USER INTERFACE:**

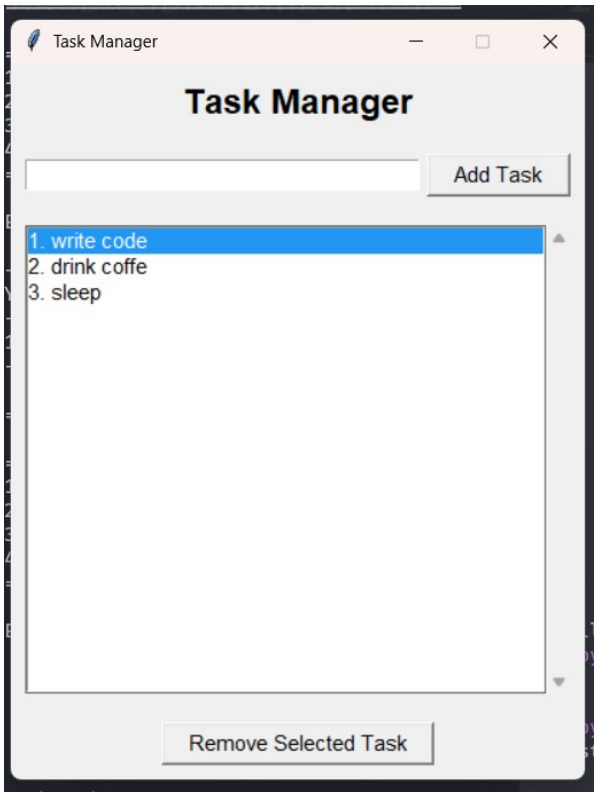
PYTHON CODE:

```
import tkinter as tk
from tkinter import messagebox
class TaskManagerGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Task Manager")
        self.root.geometry("400x500")
        self.root.resizable(False, False)
        self.tasks = []
        self.create_widgets()
    def create_widgets(self):
        title_label = tk.Label(
            self.root,
            text="Task Manager",
            font=("Arial", 18, "bold"),
              pady=10)
        title_label.pack()
        input_frame = tk.Frame(self.root)
        input_frame.pack(pady=10, padx=10, fill=tk.X)
        self.task_entry = tk.Entry(
            input_frame,
            font=("Arial", 12),
            width=25)
        self.task_entry.pack(side=tk.LEFT, padx=(0, 5), fill=tk.X, expand=True)
        self.task_entry.bind("<Return>", lambda e: self.add_task())
        add_button = tk.Button(
            input_frame,
            text="Add Task",
            font=("Arial", 11),command=self.add_task,width=10)
```

```
    add_button.pack(side=tk.LEFT)
    list_frame = tk.Frame(self.root)
    list_frame.pack(pady=10, padx=10, fill=tk.BOTH, expand=True)
    scrollbar = tk.Scrollbar(list_frame)
    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
    self.task_listbox = tk.Listbox(
        list_frame,
        font=("Arial", 11),
        yscrollcommand=scrollbar.set,
        selectmode=tk.SINGLE,
        activestyle="none",
        selectbackground="#2196F3",
        selectforeground="white"
    )
    self.task_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    scrollbar.config(command=self.task_listbox.yview)
    button_frame = tk.Frame(self.root)
    button_frame.pack(pady=10)
    remove_button = tk.Button(
        button_frame,
        text="Remove Selected Task",
        font=("Arial", 11),
        command=self.remove_task,
        width=20
    )
    remove_button.pack()
def add_task(self):
    task = self.task_entry.get().strip()
    if not task:
        messagebox.showwarning("Empty Task", "Please enter a task description.")
        return
    self.tasks.append(task)
    self.refresh_listbox()
    self.task_entry.delete(0, tk.END)
```

```
    def remove_task(self):
        try:
            selected_index = self.task_listbox.curselection()[0]
            self.tasks.pop(selected_index)
            self.refresh_listbox()
        except IndexError:
            messagebox.showwarning("No Selection", "Please select a task to remove.")

    def refresh_listbox(self):
        self.task_listbox.delete(0, tk.END)
        for index, task in enumerate(self.tasks, start=1):
            self.task_listbox.insert(tk.END, f"{index}. {task}")
def main():
    root = tk.Tk()
    app = TaskManagerGUI(root)
    root.mainloop()
if __name__ == "__main__":
    main()
```

OUTPUT:



3.**VOICE USER INTERFACE:**

PYTHON CODE:

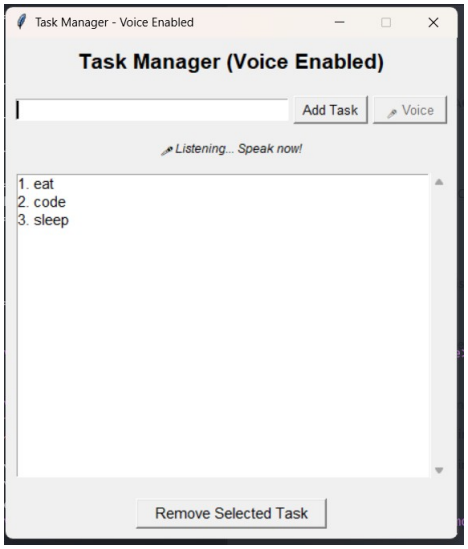
```
import tkinter as tk

from tkinter import messagebox

import speech_recognition as sr

import threading

class TaskManagerVoiceGUI:

    def __init__(self, root):

        self.root = root

        self.root.title("Task Manager - Voice Enabled")

        self.root.geometry("450x500")

        self.root.resizable(False, False)

        self.tasks = []

        self.recognizer = sr.Recognizer()

        self.is_listening = False

        self.create_widgets()

    def create_widgets(self):

        title_label = tk.Label(

            self.root,

            text="Task Manager (Voice Enabled)",

            font=("Arial", 16, "bold"),

            pady=10

        )

        title_label.pack()

        input_frame = tk.Frame(self.root)

        input_frame.pack(pady=10, padx=10, fill=tk.X)

        self.task_entry = tk.Entry(
```

```
    input_frame,
    font=("Arial", 12),
    width=20
)
self.task_entry.pack(side=tk.LEFT, padx=(0, 5), fill=tk.X, expand=True)
self.task_entry.bind("<Return>", lambda e: self.add_task())


add_button = tk.Button(
    input_frame,
    text="Add Task",
    font=("Arial", 10),
    command=self.add_task,
    width=8
)
add_button.pack(side=tk.LEFT, padx=(0, 5))
self.mic_button = tk.Button(
    input_frame,
    text="🎤 Voice",
    font=("Arial", 10),
    command=self.start_voice_input,
    width=8
)
self.mic_button.pack(side=tk.LEFT)
self.listening_label = tk.Label(
    self.root,
    text="",
    font=("Arial", 9, "italic"),
```

```
        pady=5
    )
    self.listening_label.pack()
    list_frame = tk.Frame(self.root)
    list_frame.pack(pady=10, padx=10, fill=tk.BOTH, expand=True)
    scrollbar = tk.Scrollbar(list_frame)
    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
    self.task_listbox = tk.Listbox(
        list_frame,
        font=("Arial", 11),
        yscrollcommand=scrollbar.set,
        selectmode=tk.SINGLE,
        activestyle="none"
    )
    self.task_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    scrollbar.config(command=self.task_listbox.yview)
    button_frame = tk.Frame(self.root)
    button_frame.pack(pady=10)
    remove_button = tk.Button(
        button_frame,
        text="Remove Selected Task",
        font=("Arial", 11),
        command=self.remove_task,
        width=20
    )
    remove_button.pack()
def add_task(self, task_text=None):
```

```
    if task_text is None:
        task = self.task_entry.get().strip()
    else:
        task = task_text.strip()
    if not task:
        messagebox.showwarning("Empty Task", "Please enter a task description.")
        return
    self.tasks.append(task)
    self.refresh_listbox()
    self.task_entry.delete(0, tk.END)
def remove_task(self):
    try:
        selected_index = self.task_listbox.curselection()[0]
        self.tasks.pop(selected_index)
        self.refresh_listbox()
    except IndexError:
        messagebox.showwarning("No Selection", "Please select a task to remove.")
def refresh_listbox(self):
    self.task_listbox.delete(0, tk.END)
    for index, task in enumerate(self.tasks, start=1):
        self.task_listbox.insert(tk.END, f"{index}. {task}")
def start_voice_input(self):
    if self.is_listening:
        return
    thread = threading.Thread(target=self.listen_for_task, daemon=True)
    thread.start()
def listen_for_task(self):
```

```
        self.is_listening = True
        self.mic_button.config(state=tk.DISABLED)
        self.listening_label.config(text="🎤 Listening... Speak now!")

        try:
            with sr.Microphone() as source:
                self.recognizer.adjust_for_ambient_noise(source, duration=0.5)
                audio = self.recognizer.listen(source, timeout=5, phrase_time_limit=10)
                self.listening_label.config(text="Processing...")
                task_text = self.recognizer.recognize_google(audio)
                self.root.after(0, lambda: self.add_task(task_text))
                self.root.after(0, lambda: self.listening_label.config(text=f"✓ Recognized: {task_text}"))
        except sr.WaitTimeoutError:
            self.root.after(0, lambda: self.listening_label.config(text="No speech detected. Please try again."))
        except sr.UnknownValueError:
            self.root.after(0, lambda: self.listening_label.config(text="Could not understand audio. Please try again."))
        except sr.RequestError as e:
            self.root.after(0, lambda: messagebox.showerror("Error", f"Speech recognition service error: {e}"))
            self.root.after(0, lambda: self.listening_label.config(text=""))
        except Exception as e:
            self.root.after(0, lambda: messagebox.showerror("Error", f"Microphone error: {e}"))
            self.root.after(0, lambda: self.listening_label.config(text=""))
        finally:
            self.is_listening = False
            self.root.after(0, lambda: self.mic_button.config(state=tk.NORMAL))
            self.root.after(3000, lambda: self.listening_label.config(text=""))
```

```
def main():
    root = tk.Tk()
    app = TaskManagerVoiceGUI(root)
    root.mainloop()
if __name__ == "__main__":
    main()
```

OUTPUT: